

Node.js 개발환경 구성 및 소개

□ Node.js 설치 및 개발환경 구성

○ Node.js 설치

- 공식 사이트: <https://nodejs.org>
- LTS(Long Term Support) 버전 추천

Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

Get Node.js®

Get security support

for EOL Node.js versions



Node.js is proudly supported by the partners above and more.

Create an HTTP Server Write Tests Read and Hash a File Streams Pipeline

```
1 // server.mjs
2 import { createServer } from 'node:http';
3
4 const server = createServer((req, res) => {
5   res.writeHead(200, { 'Content-Type': 'text/plain' });
6   res.end('Hello World!\n');
7 });
8
9 // starts a simple http server locally on port 3000
10 server.listen(3000, '127.0.0.1', () => {
11   console.log('Listening on 127.0.0.1:3000');
12 });
13
14 // run with `node server.mjs`
```

JavaScript

</> 클립보드에 복사

Learn more what Node.js is able to offer with our Learning materials.

- 설치 확인

- `node -v`
- `npm -v`

○ Visual Studio Code 설치

- 공식 사이트: <https://code.visualstudio.com>

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



↓ Windows

Windows 10, 11

User Installer [x64](#) [Arm64](#)
System
Installer [x64](#) [Arm64](#)
.zip [x64](#) [Arm64](#)
CLI [x64](#) [Arm64](#)



↓ .deb

Debian, Ubuntu

↓ .rpm

Red Hat, Fedora, SUSE

.deb [x64](#) [Arm32](#) [Arm64](#)
.rpm [x64](#) [Arm32](#) [Arm64](#)
.tar.gz [x64](#) [Arm32](#) [Arm64](#)
Snap [Snap Store](#)
CLI [x64](#) [Arm32](#) [Arm64](#)



↓ Mac

macOS 10.15+

.zip [Intel chip](#) [Apple silicon](#) [Universal](#)
CLI [Intel chip](#) [Apple silicon](#)

By downloading and using Visual Studio Code, you agree to the [license terms](#) and [privacy statement](#).

- Node.js 개발에 최적화

- JavaScript, TypeScript 기본 지원
- npm, 터미널, Git 통합 기능 내장

- 다양한 확장 프로그램

- ESLint, Prettier, Live Server, GitLens 등으로 개발 생산성 향상

- 자동 완성 및 오류 표시 기능

- IntelliSense로 빠른 코드 작성
- 실시간 문법 오류 체크로 실수 방지

- 터미널 내장

- VS Code 안에서 바로 **node**, **npm** 명령어 실행 가능

□ Node.js란 무엇인가?

○ 정의

Node.js는 Chrome V8 JavaScript 엔진 위에서 동작하는 서버 사이드 JavaScript 런타임입니다.

- JavaScript를 브라우저가 아닌 서버에서도 실행할 수 있게 해줌
- 비동기 이벤트 기반 아키텍처
- 싱글 스레드지만 높은 동시성 처리 가능

○ 왜 Node.js를 사용할까?

- JavaScript를 클라이언트와 서버에서 모두 사용 → 생산성 향상
- 빠른 속도 (V8 엔진 + 논블로킹 I/O)
- npm 생태계가 매우 활발함 (오픈소스 모듈 수 세계 1위)
- 실시간 애플리케이션에 적합 (채팅, 알림 등)

○ Node.js의 특징

- 이벤트 기반(Event-driven): 요청이 들어오면 이벤트 큐에 저장하고, 이벤트 루프가 처리
- 논블로킹 I/O(Non-blocking I/O): 파일 읽기, DB 요청 등이 끝날 때까지 기다리지 않음
- 싱글 스레드(Single-thread): 하나의 스레드로 동작하지만 내부적으로는 libuv로 멀티 스레드

처리 가능

□ Node.js의 실행 구조

○ 구성 요소

사용자의 요청



Node.js 엔진

JavaScript 코드 실행

이벤트 루프

libuv (C++)

사용자의 요청에 의해 코드 실행
오래 걸리는 작업은 이벤트 루프로 넘김

오래 걸리는 작업을 백그라운드(libuv)로 넘김
끝난 작업 체크

파일 읽기, 네트워크 요청 등 오래 걸리는 작업
처리

멀티 스레드로 병렬 처리 → 작업 후 이벤트
루트에게 알림

○ V8 엔진이란?

Google Chrome에서 사용하는 JavaScript 엔진

- JavaScript를 기계어로 변환하여 빠르게 실행

[실습]

"Hello Node.js!" 출력하기

1. 프로젝트 생성

```
mkdir myapp && cd myapp
```

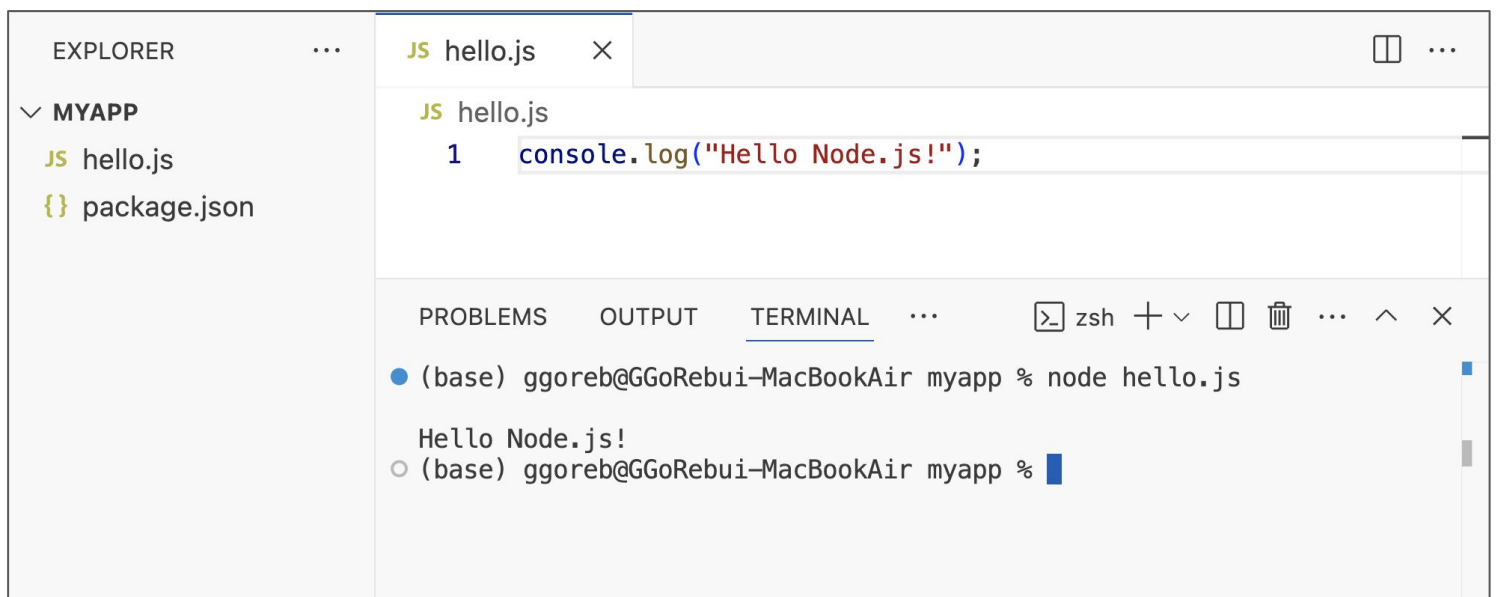
```
npm init -y
```

2. 코드 작성 - hello.js

```
console.log("Hello Node.js!");
```

3. 동작 확인

```
node hello.js
```



NPM (Node Package Manager)

□ NPM

○ 설명

- 외부 라이브러리를 설치하고 의존성을 관리하는 도구
- 프로젝트마다 설치된 모듈 정보를 `package.json`에 기록

○ package.json 주요 항목

- name, version, scripts, dependencies 등

```
{
  "name": "myapp",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "type": "module"
}
```

[실습]

chalk 모듈 설치 및 색상 출력

1. chalk 모듈 설치

npm install chalk

2. 코드 작성 - color.js

import chalk from 'chalk';

console.log(chalk.green('✅ 성공적으로 설치되었습니다!'));

3. 동작 확인

node color.js

✅ 성공적으로 설치되었습니다!

□ Node.js 내장 모듈 실습

○ fs 모듈 (파일 시스템) - fs-test.js

// 파일에 문자열을 쓰고, 다시 읽어서 출력하는 예제

```
import fs from 'node:fs';
```

// 파일에 동기적으로 텍스트를 씁니다 (파일이 없으면 새로 생성)

```
fs.writeFileSync('test.txt', '파일에 쓰기');
```

// 파일을 동기적으로 읽어옵니다

```
const data = fs.readFileSync('test.txt', 'utf8');
```

```
console.log(data); // → '파일에 쓰기'
```

○ path 모듈 (경로 처리) - [path-test.js](#)

// 경로 관련 기능을 사용하는 예제

```
import path from 'path';
```

```
const filepath = import.meta.dirname;
```

```
const filename = import.meta.filename;
```

```
const executepath = path.resolve();
```

```
console.log('파일 위치:', filepath);
```

```
console.log('현재 파일:', filename);
```

```
console.log('실행 위치:', executepath);
```

○ os 모듈 (운영체제 정보) - os-test.js

// 운영체제 관련 정보를 출력하는 예제

```
import os from 'os';
```

```
console.log(os.platform()); // 현재 운영체제 플랫폼 (e.g., darwin, win32)
```

```
console.log(os.cpus()); // CPU 코어 정보 배열
```

□ 비동기 구조 알아보기

○ read-async.js (콜백 방식)

// 콜백을 이용해 파일을 비동기적으로 읽어오는 예제

```
import fs from 'fs';
```

```
fs.readFile('test.txt', 'utf8', (err, data) => {  
  if (err) throw err;  
  console.log(data); // test.txt에 들어있는 문자열 출력  
});
```

○ promise-example.js (Promise 방식)

// Promise를 생성하여 setTimeout 이후 문자열을 반환합니다

```
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => resolve('끝!'), 1000);  
});
```

// 비동기 작업이 끝난 후 실행되는 코드

```
promise.then(console.log); // 1초 뒤 '끝!' 출력
```

○ async-await.js (async/await 방식)

// 위 Promise를 async/await 방식으로 실행하는 예제

```
const promise = new Promise((resolve) => {  
  setTimeout(() => resolve('끝!'), 1000);  
});
```

```
async function start() {  
  const result = await promise; // 비동기 결과를 기다림  
  console.log(result); // '끝!'  
}
```

```
start();
```

연습문제

1. 로그 기록기 만들기 (write-log.js)

- 설명: 웹 서비스에서는 에러나 이벤트 발생 시 로그를 파일로 저장합니다.
- 요구사항
 - 현재 시각과 메시지를 문자열로 구성
 - `logs/log.txt` 파일에 한 줄씩 누적 저장
 - `logs/` 폴더가 없으면 자동 생성
- 예상 출력 (log.txt):
 - [2025-05-15 10:34:12] 서버가 시작되었습니다
 - [2025-05-15 10:34:35] 사용자가 로그인했습니다
- 힌트
 - `fs.existsSync`(폴더 존재여부 확인), `fs.mkdirSync`(폴더 생성), `fs.appendFileSync`(내용 추가)
 - `path.join()`으로 경로 조합

```
import fs from 'fs';
import path from 'path';

// 코드 작성 1) 현재 파일의 경로와 이름 가져오기
// import.meta.dirname : 현재 파일의 경로
// import.meta.filename : 현재 파일의 이름

// 코드 작성 2) logs 폴더와 log.txt 파일 경로 설정
// path.join(경로1, 경로2) : 경로를 합쳐서 새로운 경로를 생성
// path.join(경로1, 파일명)

// 코드 작성 3) logs 폴더가 있는지 확인해서 없다면 생성하기
// fs.existsSync(경로) : 파일이나 폴더가 존재하는지 확인
// fs.mkdirSync(경로) : 폴더를 생성

const now = new Date(); // 현재 일시 2026-01-01T02:05:28.641Z
const time = now.toISOString().replace('T', ' ').substring(0, 19); // 2026-01-01 02:05:28
const message = `[${time}] 서버가 시작되었습니다\n`;

// 코드 작성 4) 로그 파일에 한줄 추가하기
// fs.appendFileSync(파일, 내용) : 파일에 내용을 추가

console.log('로그가 기록되었습니다.');
```


연습문제

2. 폴더 내 파일 목록 출력기 (list-files.js)

- 설명: 파일 탐색기, 백업 도구 등은 폴더 내 파일들을 나열하는 기능이 필수입니다.

- 요구사항

- 현재 실행 경로 내 모든 파일 이름 출력
- 전체 경로가 아닌 파일 이름만 출력

- 예상 출력

파일 목록:

- data1.txt
- data2.txt
- notes.json

- 힌트

- `path.resolve()` : 현재 프로젝트가 실행된 경로(작업 디렉토리)를 절대 경로로 반환
- `fs.readdirSync(경로)` : 폴더 안의 파일/폴더 이름을 배열로 반환

```
import fs from 'fs';
import path from 'path';

// 코드 작성 1) 현재 실행 경로 가져오기
// path.resolve() : 현재 실행 경로를 절대 경로로 가져오기

// 코드 작성 2) 파일 목록 가져오기
// fs.readdirSync(경로) : 폴더 내 파일 목록을 배열로 가져오기

console.log('파일 목록:');
files.forEach(file => {
  console.log(`- ${file}`);
});
```

연습문제

3. 텍스트 파일 복사 도구 (`copy-text.js`)

- 설명: 서버 유지 보수 시 설정 파일 백업이 자주 발생합니다.

파일을 읽어서 다른 이름으로 저장하면 간단한 백업 도구를 만들 수 있습니다.

- 요구사항

- `original.txt`의 내용을 `backup.txt`로 복사
- 기존에 `backup.txt`가 있으면 덮어쓰

- 힌트:

- `fs.readFileSync(파일경로, 'utf8')`: 파일을 동기적으로 읽어서 문자열로 반환
- `fs.writeFileSync(파일경로, 내용)`: 파일에 내용을 씀 (파일이 있으면 덮어쓰고, 없으면 새로 생성)
- `path.join()`으로 경로 조합

```
import fs from 'fs';
import path from 'path';

// 코드 작성 1) original.txt, backup.txt의 경로 만들기
// import.meta.dirname : 현재 파일이 있는 폴더의 경로
// path.join(경로1, 파일명) : 경로를 합쳐서 새로운 경로 생성

// 코드 작성 2) original.txt 파일 읽어오기
// fs.readFileSync(파일경로, 'utf8') : 파일을 문자열로 읽어옴

// 코드 작성 3) 읽어온 내용을 backup.txt에 쓰기
// fs.writeFileSync(파일경로, 내용) : 파일에 내용을 저장 (기존 파일 있으면 덮어쓰)

console.log('복사가 완료되었습니다.');
```

파일 위치 찾기

□ 위치 찾기가 중요한 이유

- 파일 다운로드, 업로드, 정적 파일 응답 등 자주 사용되는 기능
- 다양한 위치 기준이 섞여 있어서 혼란 발생
 - `res.download(path.join(__dirname, 'public', 'example.txt'));`
 - `res.download(path.resolve('public', 'example.txt'));`

□ `process.cwd()` - 작업 디렉토리 기준 (nodejs가 실행된 위치)

```
console.log(process.cwd());
```

- 프로젝트 루트 디렉토리에서 실행한 경우 → 프로젝트 루트 디렉토리

```
● ggoreb@GGoRebui-MacBookAir myapp % pwd
/Users/ggoreb/study/myapp
● ggoreb@GGoRebui-MacBookAir myapp % node ./01/test.js
/Users/ggoreb/study/myapp
○ ggoreb@GGoRebui-MacBookAir myapp %
```

- 하위 디렉토리에서 실행한 경우 → 하위 디렉토리

```
● ggoreb@GGoRebui-MacBookAir 01 % pwd
/Users/ggoreb/study/myapp/01
● ggoreb@GGoRebui-MacBookAir 01 % node test.js
/Users/ggoreb/study/myapp/01
○ ggoreb@GGoRebui-MacBookAir 01 %
```

□ `import.meta` - 코드가 실행되는 파일 기준

```
console.log(import.meta.dirname); // 파일의 위치
```

```
console.log(import.meta.filename); // 파일의 위치 + 파일명
```

```
console.log(import.meta.url); // 파일 URI
```

```
● ggoreb@GGoRebui-MacBookAir 01 % node test.js
/Users/ggoreb/study/myapp/01
/Users/ggoreb/study/myapp/01/test.js
file:///Users/ggoreb/study/myapp/01/test.js
○ ggoreb@GGoRebui-MacBookAir 01 %
```

□ `path.join()` - 경로를 결합하여 새로운 경로 생성 (상대 경로)

```
import path from 'path';
```

```
console.log(path.join("public", "sample.txt"));
```

```
● ggoreb@GGoRebui-MacBookAir 01 % node test.js
  public/sample.txt
○ ggoreb@GGoRebui-MacBookAir 01 %
```

□ `path.resolve()` - 작업 디렉토리 + 경로 (절대 경로)

```
import path from 'path';
```

```
console.log(path.resolve());
```

```
console.log(path.resolve("public", "sample.txt"));
```

```
console.log(path.resolve(import.meta.dirname, "public", "sample.txt"));
```

```
● ggoreb@GGoRebui-MacBookAir 01 % node test.js
  /Users/ggoreb/study/myapp/01
  /Users/ggoreb/study/myapp/01/public/sample.txt
  /Users/ggoreb/study/myapp/01/public/sample.txt
○ ggoreb@GGoRebui-MacBookAir 01 %
```

□ 정리

○ 최신 버전 (v20.6.0 이상)

- 실행 위치: `path.resolve()` 또는 `process.cwd()`
- 현재 파일: `import.meta.filename`
- 현재 파일 위치: `import.meta.dirname`

○ 이전 버전 (v20.6.0 미만)

- 실행 위치: `path.resolve()` 또는 `process.cwd()`
- 현재 파일: `fileURLToPath(import.meta.url)`
- 현재 파일 위치: `path.dirname(fileURLToPath(import.meta.url))`

○ (OS에 맞는) 상대 경로 생성: `path.join()`

HTTP 서버 만들기

□ HTTP 모듈

- Node.js의 기본 내장 모듈인 `http`를 사용하여 서버 구현 가능
- `http.createServer()` 함수로 요청 처리 함수 생성
 - `.listen(port)` 함수로 실행 시작

[실습]

가장 기본적인 HTTP 서버 만들기

1. 코드 작성 - server-basic.js

```
import http from 'http';
```

```
const server = http.createServer((req, res) => {  
  res.statusCode = 200; // 200 OK  
  res.setHeader('Content-Type', 'text/plain');  
  res.end('Hello, HTTP Server!');  
});
```

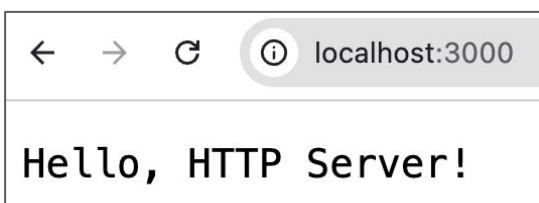
```
server.listen(3000, () => {  
  console.log('서버 실행 중: http://localhost:3000');  
});
```

2. 동작 확인

node server-basic.js

```
ggoreb@GGoRebui-MacBookAir 01 % node server-basic.js  
서버 실행 중: http://localhost:3000
```

(브라우저) `http://localhost:3000` 접속



라우팅 처리하기

□ Routing

- 라우팅이란 **URL** 경로에 따라 서로 다른 응답을 하는 것
- if문 또는 **switch**문으로 분기 처리 가능
- 하나의 서버가 여러 페이지를 구분하여 처리하려면 **URL** 경로를 기준으로 응답을 나눠야 함
- 실무에서는 **REST API**나 정적 페이지 응답에도 반드시 사용

[실습]

if문으로 라우팅 처리

1. 코드 작성 - server-routing.js

```
import http from 'http';

const server = http.createServer((req, res) => {
  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end('홈입니다');
  } else if (req.url === '/about') {
    res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end('소개 페이지입니다');
  } else {
    res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end('페이지를 찾을 수 없습니다');
  }
});

server.listen(3000, () => {
  console.log('http://localhost:3000 에서 실행 중');
});
```

2. 동작 확인

node server-routing.js

http://localhost:3000

홈입니다

http://localhost:3000/about

소개 페이지입니다

http://localhost:3000/xyz

페이지를 찾을 수 없습니다

HTML 파일로 응답하기

□ 개념

- HTML, CSS, JS 같은 정적 파일을 읽어서 클라이언트에 전달하는 동작
- 실제 서비스에서는 단순한 문자열이 아니라 HTML 문서를 통해 응답
- 브라우저는 파일의 MIME 타입(Content-Type)을 통해 HTML인지 구분

[실습]

index.html 파일을 읽어서 브라우저에 응답

1. 코드 작성 - server-html.js

```
import http from 'http';
import fs from 'fs';
import path from 'path';

const server = http.createServer((req, res) => {
  if (req.url === '/') {
    const filePath = path.join(import.meta.dirname, 'index.html');

    fs.readFile(filePath, (err, data) => {
      if (err) {
        res.writeHead(500, { 'Content-Type': 'text/plain' });
        res.end('서버 오류 발생');
      } else {
        res.writeHead(200, { 'Content-Type': 'text/html' });
        res.end(data);
      }
    });
  }
});

server.listen(3000, () => {
  console.log('HTML 서버 실행 중: http://localhost:3000');
});
```

2. 코드 작성 - index.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head><meta charset="UTF-8"><title>홈페이지</title></head>
```

```
<body><h1>환영합니다!</h1></body>
```

```
</html>
```

3. 동작 확인

```
node server-html.js
```

환영합니다!

GET 요청 데이터 처리

□ 개념

- GET은 웹 요청의 기본이며 주로 브라우저에서 링크 클릭이나 주소창 입력 시 사용
- 요청 데이터는 URL 뒤에 ? 기호를 사용해 쿼리 문자열로 전달
 - `http://localhost:3000?name=joyvit&age=25`
- 쿼리 문자열은 ? 이후에 `key=value` 형식으로 나열되고 &로 구분
- 일반적으로 서버의 데이터를 변경하지 않는 '조회(Read)' 용도로 사용

[실습]

GET 방식으로 전달된 쿼리 문자열을 분석하여 응답

1. 코드 작성 - server-get.js

```
import http from 'http';
```

```
const server = http.createServer((req, res) => {
  if (req.method === 'GET') {
    // req.url은 주소창의 뒷부분만 오므로 표준 URL 객체를 만들기 위해 베이스 주소(호스트)를
    합쳐줍니다
    const baseUrl = `http://${req.headers.host}`;
    const url = new URL(req.url, baseUrl);

    const pathname = url.pathname;
    const query = url.searchParams;

    if (pathname === '/') {
      const name = query.get('name') || '익명';
      const age = query.get('age') || '알 수 없음';

      res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf8' });
      res.end(`이름: ${name}, 나이: ${age}`);
    } else {
      res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf8' });
      res.end('페이지를 찾을 수 없습니다');
    }
  } else {
    res.writeHead(405, { 'Content-Type': 'text/plain; charset=utf8' });
    res.end('GET 요청만 허용됩니다');
  }
});

server.listen(3000, () => {
  console.log('GET 서버 실행 중: http://localhost:3000');
});
```

2. 동작 확인

node server-get.js

(API Tester)

METHOD

GET

SCHEME :// HOST [":" PORT] [PATH ["?" QUERY]]

http://localhost:3000?name=park&age=30

length: 38 byte(s)

Send

QUERY PARAMETERS

☒ name

=

park

×

⋮

☒ age

=

30

×

⋮

+ Add query parameter

🗑

▶ BODY

이름: park, 나이: 30

POST 요청 데이터 처리

□ 개념

- POST는 클라이언트가 서버로 데이터를 전송하는 방식
- 서버는 데이터를 조각으로 받아 처리하며, 폼 전송, 로그인 등에 활용됨
- 파일 업로드, 회원가입, 로그인, 댓글 작성 등의 핵심 기능

[실습]

POST 방식으로 전송된 데이터를 읽고 그대로 응답

1. 코드 작성 - server-post.js

```
import http from 'http';
```

```
const server = http.createServer((req, res) => {
```

```
  if (req.method === 'POST') {
```

```
    let body = '';
```

```
    // post 요청 데이터는 조각(chunk) 단위로 쪼개어져서 도착되므로 하나로 합침
```

```
    req.on('data', chunk => {
```

```
      body += chunk;
```

```
    });
```

```
    req.on('end', () => {
```

```
      res.writeHead(200, { 'Content-Type': 'text/plain' });
```

```
      res.end(`받은 데이터: ${body}`);
```

```
    });
```

```
  } else {
```

```
    res.writeHead(405, { 'Content-Type': 'text/plain; charset=utf8' });
```

```
    res.end('POST 요청만 허용됩니다');
```

```
  }
```

```
});
```

```
server.listen(3000, () => {
```

```
  console.log('POST 서버 실행 중: http://localhost:3000');
```

```
});
```

2. 동작 확인

node server-post.js

(API Tester)

METHOD: **POST** SCHEME :// HOST [":" PORT] [PATH ["?" QUERY]] length: 21 byte(s)

▶ QUERY PARAMETERS

HEADERS [?]

☒ Content-Type : application/x-www-form-urlencoded ×

BODY [?] ☐ multipart/form-data

☒ id [Text] = abcd ×

☒ pw [Text] = 1234 ×

▶ BODY [?]

받은 데이터: **id=abcd&pw=1234**

연습문제

1. 모든 쿼리 문자열을 JSON 형태로 응답하기 (query-echo.js)

클라이언트가 URL에 담아 보낸 쿼리 문자열을 모두 읽어서, JSON 형태로 만들어 응답하는 서버를 작성합니다.

- 설명

- GET 요청은 URL 뒤에 `?key=value&key2=value2` 형태로 데이터를 전달합니다.
- 요청을 처리한 뒤 JSON 문자열로 바꿔 응답하는 것은 API 서버에서 아주 흔한 패턴입니다.

- 요구사항

- 클라이언트가 URL로 전달한 모든 쿼리 문자열을 JSON 형태로 만들어 응답
- **GET 요청**에 대해서만 응답

- 예시

요청 → `/?city=seoul&weather=sunny`

응답 → `{"city": "seoul", "weather": "sunny"}`

```
import http from 'http';
```

```
const server = http.createServer((req, res) => {  
  if (req.method === 'GET') {  
    const baseUrl = `http://${req.headers.host}`;  
    const url = new URL(req.url, baseUrl);  
    const query = url.searchParams;  
  
    const queryObj = {};  
    for (const [key, value] of query) {  
      queryObj[key] = value; // queryObj에 key-value 쌍 추가  
    }  
  
    // 코드 작성 1) queryObj를 JSON 문자열로 변환하기  
    // JSON.stringify(객체) : 객체를 JSON 형태의 문자열로 변환  
  
    // 코드 작성 2) 응답 헤더를 설정하고 (Content-Type: application/json), 응답 보내기  
    // res.writeHead(200, { 'Content-Type': ... })  
    // res.end(내용) : 응답 종료 및 전송  
  
  } else {  
    res.writeHead(405, { 'Content-Type': 'text/plain; charset=utf8' });  
    res.end('GET 요청만 허용됩니다');  
  }  
});
```

```
URL {  
  href: 'http://localhost:3000/?key=value&key2=value2',  
  origin: 'http://localhost:3000',  
  protocol: 'http:',  
  username: '',  
  password: '',  
  host: 'localhost:3000',  
  hostname: 'localhost',  
  port: '3000',  
  pathname: '/',  
  search: '?key=value&key2=value2',  
  searchParams: URLSearchParams { 'key' => 'value', 'key2' => 'value2' },  
  hash: ''  
}
```

```
server.listen(3000, () => {  
  console.log('서버 실행 중: http://localhost:3000');  
});
```

연습문제

2. 인사 문장 만들기 (page-greeting.js)

클라이언트가 폼 데이터로 보낸 이름을 받아서 인사 문장을 만들어 응답하는 서버를 작성합니다.

- 설명

- 서버가 **POST**로 전달된 데이터를 읽어서 그 안의 값을 활용하는 것은 로그인, 회원가입, 댓글 작성 등

실무에서 매우 자주 쓰이는 패턴입니다.

- 이번엔 받은 데이터를 그대로 돌려주는 것을 넘어 그 값을 이용해 새로운 문장을 만들어봅니다.

- 요구사항

- URL이 **/hello**인 경우, **name** 쿼리 값을 받아서 인사 문장을 만든 후 응답
- **POST 요청**에 대해서만 응답
- **name**이 없으면 "누구신가요?" 라고 응답

- 예시

요청 → **/hello** (formdata → name=joyvit)

응답 → 안녕하세요, joyvit님!

요청 → **/hello** (formdata → 없음)

응답 → 누구신가요?

연습문제

2. 인사 문장 만들기 (page-greeting.js)

```
import http from 'http';

const server = http.createServer((req, res) => {
  // POST 방식 확인: req.method == 'POST'
  // URL 확인: req.url == '/hello'
  if ( /* 코드 작성 1) POST 요청이면서 url이 '/hello'인 경우만 처리하도록 조건 작성하기 */ ) {
    let body = "";

    req.on('data', chunk => {
      body += chunk;
    });

    req.on('end', () => {
      // body는 'name=joyvit' 같은 형태의 문자열입니다.
      // URLSearchParams는 'key=value&key2=value2' 형태의 문자열을 쉽게 다루게 해줍니다.
      const params = new URLSearchParams(body);
      const name = params.get('name'); // 값이 없으면 null 반환

      // 코드 작성 2) name이 없으면(null이면) '누구신가요?' 응답하기
      // 코드 작성 3) name이 있으면 `안녕하세요, ${name}님!` 형태로 응답하기
      // res.writeHead(200, { 'Content-Type': ... }), res.end(내용) 사용

    });
  } else {
    res.writeHead(405, { 'Content-Type': 'text/plain; charset=utf8' });
    res.end('허용되지 않은 요청입니다');
  }
});

server.listen(3000, () => {
  console.log('서버 실행 중: http://localhost:3000');
});
```

상태 코드와 에러 처리

□ HTTP 상태 코드

○ 개념

- HTTP 상태 코드는 클라이언트에게 요청 처리 결과를 숫자로 알려주는 약속된 규칙
- 상태 코드는 3자리 숫자로 구성되며, 처리 결과를 요약해서 전달함

○ 코드 종류

- 200번대: 정상 처리 (성공)
- 400번대: 클라이언트 오류 (잘못된 요청 등)
- 500번대: 서버 오류 (예외, 처리 불가 등)

□ try-catch로 예외 처리하기

○ 개념

- 서버 코드 실행 중 문제가 생기면 Node.js는 예외를 발생시킴 (throw)
- 서버가 종료되지 않도록 try...catch 문으로 예외 상황을 감싸고 적절한 상태 코드와 메시지를 클라이언트에 응답해야 함

[실습]

try-catch로 에러 처리 및 상태 코드 지정

1. 코드 작성 - status-error.js

```
import http from 'http';
```

```
const server = http.createServer((req, res) => {  
  try {  
    if (req.url === '/error') throw new Error('강제 에러');  
  
    res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf8' });  
    res.end('정상 작동');  
  } catch (err) {  
    res.writeHead(500, { 'Content-Type': 'text/plain; charset=utf8' });  
    res.end('서버 내부 오류');  
  }  
});
```

```
server.listen(3000, () => {  
  console.log('http://localhost:3000 서버 실행 중');  
});
```

2. 동작 확인

```
node status-error.js
```

(브라우저) <http://localhost:3000/error> 접속

연습문제

1. 404 상태 코드 응답하기 (client-error.js)

요청 경로가 정해진 경로가 아닐 때 "페이지를 찾을 수 없다"는 뜻의 상태 코드로 응답하는 서버를 작성합니다.

- 설명

- 실무에서는 존재하지 않는 페이지에 접속했을 때 단순히 문구만 보여주는 게 아니라 브라우저와 API 클라이언트가 알아볼 수 있도록 **상태 코드**로도 결과를 알려줘야 합니다.
- **404**는 "요청한 자원을 찾을 수 없음"을 뜻하는 대표적인 클라이언트 오류 코드입니다.

- 요구사항:

- 요청 경로가 **/hello**가 아니면 **404 Not Found** 상태 코드로 응답
- **/hello**인 경우에는 정상적으로 **200** 응답

- 힌트

- **res.writeHead(404)** 사용

- 예시

요청 → **/hello**

응답 → **200**, 안녕하세요

요청 → **/아무거나**

응답 → **404**, 페이지를 찾을 수 없습니다.

```
import http from 'http';

const server = http.createServer((req, res) => {
  // 코드 작성 1) req.url이 '/hello'인 경우와 아닌 경우를 나누기

  // 코드 작성 2) 200 상태 코드로 정상 응답하기
  // res.writeHead(200, { 'Content-Type': ... }), res.end(내용)

  // 코드 작성 3) 404 상태 코드로 응답하기
  // res.writeHead(404, { 'Content-Type': ... })
  // res.end('페이지를 찾을 수 없습니다')
})

});

server.listen(3000, () => {
  console.log('서버 실행 중: http://localhost:3000');
});
```

연습문제

2. 400 상태 코드 응답하기 (`validate.js`)

클라이언트가 보낸 값이 올바르지 않은 경우 "요청 자체가 잘못되었다"는 뜻의 상태 코드로 응답합니다.

- 설명

- **404**가 "그런 페이지가 없다"는 뜻이라면 **400**은 "요청은 받았지만 값이 잘못됐다"는 뜻입니다.
- 숫자를 받아야 하는 곳에 문자가 들어오면 서버는 이를 걸러내고 클라이언트에게 알려줘야 합니다.

- 요구사항

- `/add?x=10&y=abc`처럼 숫자가 아닌 값을 받으면 **400 Bad Request**로 처리
- **GET 요청**에 대해서만 응답
- 정상일 경우 **200 OK**로 두 수를 더한 결과 출력

- 힌트

- `isNaN(Number(x))` 활용
- `res.writeHead(400)` 사용

- 예시

요청 → `/add?x=10&y=5`

응답 → 200, 결과: 15

요청 → `/add?x=10&y=abc`

응답 → 400, x, y는 숫자여야 합니다.

연습문제

2. 400 상태 코드 응답하기 (validate.js)

```
import http from 'http';

const server = http.createServer((req, res) => {
  if (req.method === 'GET') {
    const baseUrl = `http://${req.headers.host}`;
    const url = new URL(req.url, baseUrl);
    const x = url.searchParams.get('x');
    const y = url.searchParams.get('y');

    // isNaN(Number(값)) : 값이 숫자로 변환되지 않으면 true
    if ( /* 코드 작성 1) x, y가 숫자가 아니면 400 응답하기 */ ) {
      res.writeHead(400, { 'Content-Type': 'text/plain; charset=utf8' });
      res.end('x, y는 숫자여야 합니다');

    } else {
      // Number(문자열) : 문자열을 숫자로 변환
      const result = /* 코드 작성 2) x, y를 숫자로 변환해서 더한 뒤 200으로 응답하기 */
      res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf8' });
      res.end(`결과: ${result}`);
    }
  } else {
    res.writeHead(405, { 'Content-Type': 'text/plain; charset=utf8' });
    res.end('GET 요청만 허용됩니다');
  }
});

server.listen(3000, () => {
  console.log('서버 실행 중: http://localhost:3000');
});
```